

→ Exception Handling :-

→ What is an Exception:

An exception is an abnormal condition or unexpected event that occurs during the execution of a program and **disturbs** the **normal flow** of the program.

Eg:

```
int a = 10;
```

```
int b = 0;
```

```
int result = a/b ; // Exception
```

Here, dividing by zero cause an **Arithmetic Exception**.

Without exception handling, the program may terminate abnormally.

→ What is Exception Handling?

Exception Handling is a mechanism in java used to **detect, handle, and recover** from **runtime errors** so that the **normal flow** of the program can be maintained.

Java provides these keywords:

- try
- catch
- finally
- throw
- throws

→ Why Exception handling is required?

Exception handling is used to:

1. Prevent abnormal termination of the program.
2. Handle runtime ~~err~~ errors.
3. Maintain normal program flow.
4. Separate error-handling code from normal code.
5. Provide meaning full error message.
6. Improve reliability and robustness of programs.

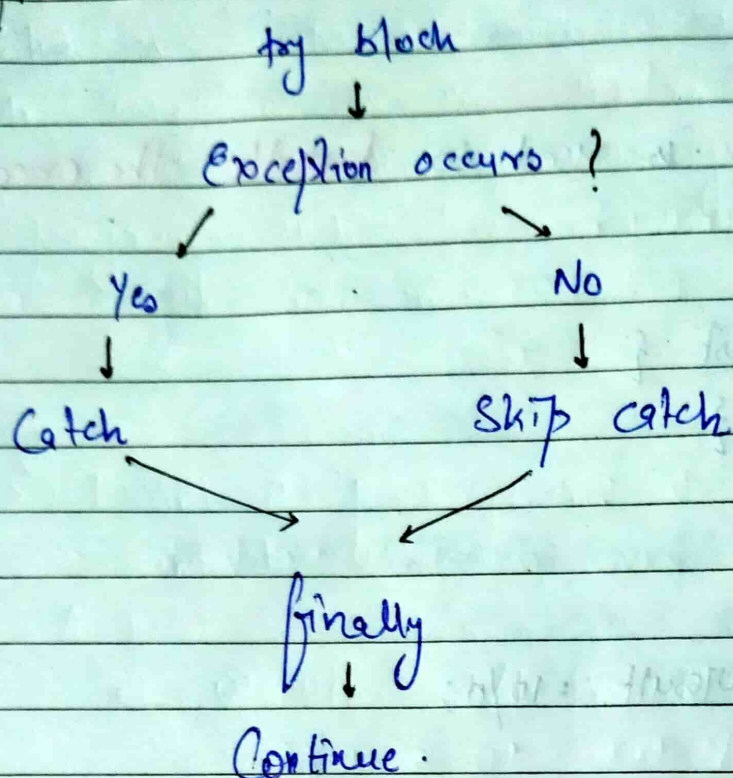
→ Structure:

```
try {
    // code that may cause exception
}
```

```
catch (ExceptionType e) {
    // code to handle exception
}
```

```
finally {
    // code that will execute after try/catch
}
```

→ Flow:



→ try block:
The try block contains the code that may generate an exception.

eg;

```
try {
```

```
    int result = 10/0;
```

```
}
```

If an exception occurs inside try, java looks for a suitable catch block.

→ The catch block is used to handle the exception.

→ Catch block:

The catch block is used to handle the exception.

Eg;

```
class Test {  
    psum {  
  
        try {  
  
            int result = 10/0;  
        }  
        catch (ArithmeticException e) {  
            Syst ("Cannot divide by zero");  
        }  
    }  
}
```

Output:

here:

- try → contains risky code
- ArithmeticException → type of exception
- e → reference to exception object
- catch → handles the exception.

→ ~~#~~ finally block:
The finally block contains code that generally executes whether an exception occurs or not.

```
try {  
    int a = 10/2;  
}  
catch (ArithmeticException e) {  
    sout("Exception occurred");  
}  
finally {  
    sout("finally block executed");  
}
```

output:

Finally block executed

Another eg;

```
try {  
    int a = 10/0;  
}  
catch (ArithmeticException e) {  
    sout("error");  
}  
finally {  
    sout("Program Completed");  
}
```

Output:

Error

Program completed

→ Main Use of finally:

It is commonly used for **cleanup** operations, such as closing

→ Multiple catch blocks:

A single try can have multiple catch blocks.

```
try {
```

```
    int[] arr = {10, 20, 30};
```

```
    sort(arr[5]);
```

```
}
```

```
catch (ArithmeticException e) {
```

```
    sort("Arithmetic error");
```

```
}
```

```
catch (ArithmeticException ArrayIndexOutOfBoundsException e) {
```

```
    sort("Array index error");
```

```
}
```

```
catch (Exception e) {
```

```
    sort("Some other error");
```

```
}
```

Output:

Array index error

Rule:

More specific exception should- should come before the general exception.

// wrong eg;

```
catch (Exception e) {  
    }  
    catch (ArithmeticException e) {  
    }
```

Because Exception can already catch ArithmeticException.

// Correct:

```
catch (ArithmeticException e) {  
    }  
    catch (Exception e) {  
    }
```

→ Throw keyword:

The Throw keyword is used to explicitly create and throw an exception.

Syntax:

```
throw new ExceptionType();
```

Eg;

```
class Test {  
    psum {  
        int age = 15;  
        if (age < 18) {
```

```
            throw new ArithmeticException("Not eligible");
```

```
        }
```

```
        cout << "Eligible";
```

```
    }
```

```
}
```

→ Throws keyword:

Throws is used in method declaration to indicate that a method may throw one or more exceptions.

Eg;

```
class Test {
```

```
    static void check() throws ArithmeticException {
```

```
        int x = 10/0;
```

```
    }
```

```
psvm {
```

```
try {
```

```
    check();
```

```
}
```

```
catch (ArithmeticException e) {
```

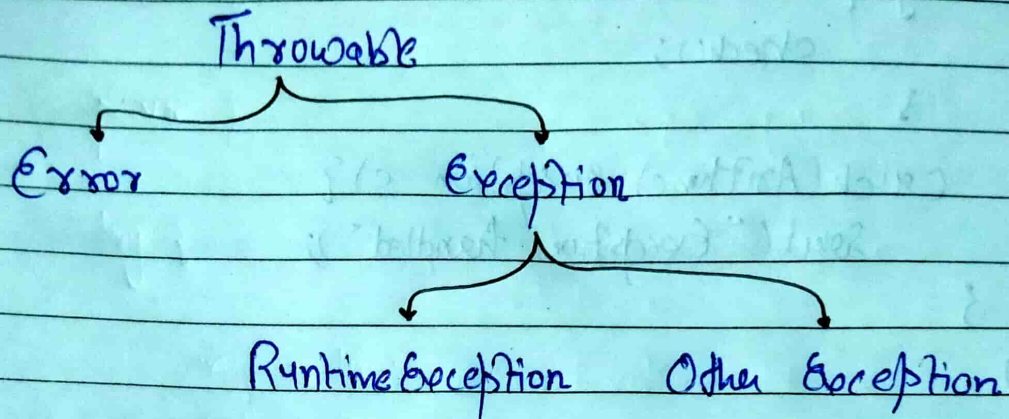
```
    sent("Exception handled");
```

```
}
```

```
}
```

```
}
```

→ Types of Exceptions:



(A) Checked Exceptions:

Checked Exception are checked by the **compiler**.

Eg;

- IOException
- SQLException
- FileNotFoundException
- ClassNotFoundException

They generally must be handled using try-catch or declared using throws.

(B) Unchecked Exceptions:

Unchecked exception occur during runtime and are subclass of **RuntimeException**.

Eg;

- ArithmeticException
- NullPointerException
- ArrayIndexOutOfBoundsException
- NumberFormatException
- ClassCastException

→ Common Exception in Java:

1. ArithmeticException:

Occurs during invalid arithmetic operations.

Eg:

```
int x = 10/0;
```

2. ArrayIndexOutOfBoundsException:

Occurs when accessing an invalid array index.

Eg:

```
int[] arr = {10, 20, 30};
```

```
System.out.println(arr[5]);
```

3. NullPointerException:

Occurs when trying to use a null reference.

Eg:

```
String name = null;
```

```
System.out.println(name.length());
```

4. NumberFormatException:

Occurs when an invalid String is converted into a number.

Eg:

```
int x = Integer.parseInt("abc");
```

Complete Eg;

```
class Test {
```

```
    psum {
```

```
        try {
```

```
            int a=10;
```

```
            int b=0;
```

```
            int result = a/b;
```

```
            sout(result);
```

```
        }
```

```
        catch(ArithmeticException e) {
```

```
            sout("Cannot divide by zero");
```

```
        }
```

```
    finally {
```

```
        sout("Finally block executed");
```

```
    }
```

```
}
```

```
}
```

Output:

Cannot divide by zero

Finally block executed

